LAB SHEET – 4

Sum.js

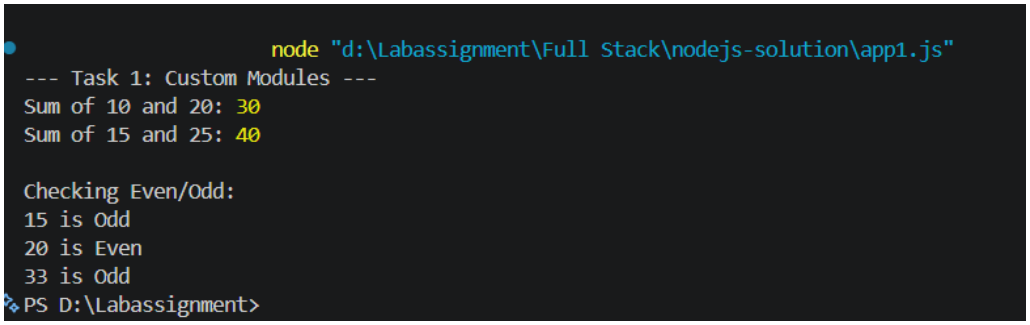
```
function calculateSum(a, b) {
   return a + b;
}
module.exports = calculateSum;
```

Even-odd.js

```
function checkEvenOdd(num) {
   if (num % 2 === 0) {
      return `${num} is Even`;
   } else {
      return `${num} is Odd`;
   }
}
module.exports = checkEvenOdd;
```

App1.js

```
const calculateSum = require('./sum.js');
const checkEvenOdd = require('./even-odd.js');
console.log('--- Task 1: Custom Modules ---');
console.log('Sum of 10 and 20:', calculateSum(10, 20));
console.log('Sum of 15 and 25:', calculateSum(15, 25));
console.log('\nChecking Even/Odd:');
console.log(checkEvenOdd(15));
console.log(checkEvenOdd(20));
console.log(checkEvenOdd(33));
```



File-ops.js

```
const fs = require('fs');
const path = require('path');
const filePath = path.join(__dirname, 'student.txt');
const newFilePath = path.join(__dirname, 'profile.txt');
console.log('--- Task 2: File System CRUD ---');
fs.writeFile(filePath, 'Name: Akshit Negi\nRoll Number: 04\n', (err) => {
    if (err) throw err;
    console.log('✓ Created student.txt and wrote name and roll number');
    fs.appendFile(filePath, 'Course: Full Stack Web Development\n', (err) => {
        if (err) throw err;
        console.log('✓ Appended course name');
        fs.readFile(filePath, 'utf8', (err, data) => {
            if (err) throw err;
            console.log('\n--- File Content ---');
            console.log(data);
            console.log('--- End of File ---\n');
            fs.rename(filePath, newFilePath, (err) => {
                if (err) throw err;
                console.log('✓ Renamed student.txt to profile.txt');
            });
        });
```

```
});
});
```

```
node "d:\Labassignment\Full Stack\nodejs-solution\file-ops.js"
--- Task 2: File System CRUD ---
✓ Created student.txt and wrote name and roll number
✓ Appended course name

--- File Content ---
Name: Akshit Negi
Roll Number: 04
Course: Full Stack Web Development

--- End of File ---

✓ Renamed student.txt to profile.txt
PS D:\Labassignment>
```

Dir-ops.js

```
const fs = require('fs');
const path = require('path');
const uploadsDir = path.join(__dirname, 'uploads');
console.log('--- Task 3: Directory Operations ---');
fs.mkdir(uploadsDir, { recursive: true }, (err) => {
    if (err) throw err;
    console.log('✓ Created uploads folder');
    const files = ['file1.txt', 'file2.txt', 'file3.txt'];
    let filesCreated = 0;
    files.forEach((file) => {
        const filePath = path.join(uploadsDir, file);
        fs.writeFile(filePath, '', (err) => {
            if (err) throw err;
            filesCreated++;
            console.log(`✓ Created ${file}`);
            if (filesCreated === files.length) {
                fs.readdir(uploadsDir, (err, fileList) => {
                    if (err) throw err;
```

```
            console.log('\n--- Files in uploads folder ---');
            fileList.forEach((f) => console.log(`  - ${f}`));
            const fileToDelete = path.join(uploadsDir, 'file2.txt');
            fs.unlink(fileToDelete, (err) => {
              if (err) throw err;
              console.log('\n✓ Deleted file2.txt');
              fs.readdir(uploadsDir, (err, fileList) => {
                if (err) throw err;
                console.log('\n--- Remaining files ---');
                fileList.forEach((f) => console.log(`  - ${f}`));
              });
            });
          });
        }
      });
    });
});
```

```
PS D:\Labassignment> node "d:\Labassignment\Full Stack\nodejs-solution\dir-ops.js"
  --- Task 3: Directory Operations ---
  ✓ Created uploads folder
  ✓ Created file1.txt
  ✓ Created file2.txt
  ✓ Created file3.txt

  --- Files in uploads folder ---
    - file1.txt
    - file2.txt
    - file3.txt

  ✓ Deleted file2.txt

  --- Remaining files ---
    - file1.txt
    - file3.txt
PS D:\Labassignment>
```

Path-demo.js

```
const path = require('path');
```

```
console.log('--- Task 4: Path Module ---');

const filePath = '/home/user/data/report.pdf';

const dirname = path.dirname(filePath);

console.log('Directory Name:', dirname);

const basename = path.basename(filePath);

console.log('Base Name:', basename);

const extname = path.extname(filePath);

console.log('Extension:', extname);

const relativePath = './documents/notes.txt';

const absolutePath = path.resolve(relativePath);

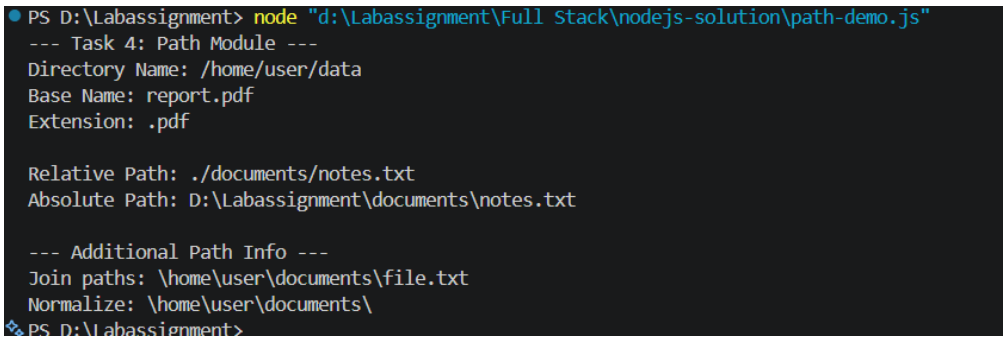
console.log('\nRelative Path:', relativePath);

console.log('Absolute Path:', absolutePath);

console.log('\n--- Additional Path Info ---');

console.log('Join paths:', path.join('/home', 'user', 'documents', 'file.txt'));

console.log('Normalize:', path.normalize('/home//user///documents/'));
```



Sys-report.js

```
const os = require('os');

console.log('--- Task 5: OS Module Report ---\n');

console.log('Platform:', os.platform());

console.log('Architecture:', os.arch());

const cpus = os.cpus();

console.log('Number of CPU Cores:', cpus.length);
```

```
const totalMemory = os.totalmem();
const freeMemory = os.freemem();
const usedMemory = totalMemory - freeMemory;
console.log('Total Memory:', (totalMemory / (1024 * 1024 * 1024)).toFixed(2), 'GB');
console.log('Free Memory:', (freeMemory / (1024 * 1024 * 1024)).toFixed(2), 'GB');
console.log('Used Memory:', (usedMemory / (1024 * 1024 * 1024)).toFixed(2), 'GB');
const uptimeSeconds = os.uptime();
const uptimeMinutes = Math.floor(uptimeSeconds / 60);
console.log('System Uptime:', uptimeMinutes, 'minutes');
console.log('Hostname:', os.hostname());
console.log('Home Directory:', os.homedir());
```

```
PS D:\Labassignment> node "d:\Labassignment\Full Stack\nodejs-solution\sys-report.js"
Platform: win32
Architecture: x64
Number of CPU Cores: 12
Total Memory: 15.68 GB
Free Memory: 6.60 GB
Used Memory: 9.08 GB
System Uptime: 244 minutes
Hostname: LAPTOP-K17ODTPG
Home Directory: C:\Users\AKSHIT NEGI
PS D:\Labassignment>
```

Order-events.js

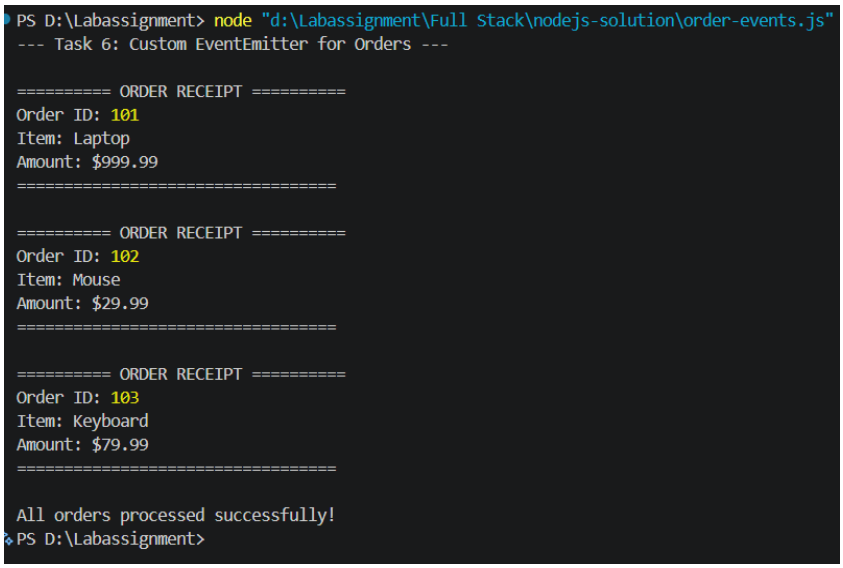
```
const EventEmitter = require('events');
console.log('--- Task 6: Custom EventEmitter for Orders ---\n');
class OrderManager extends EventEmitter {}
const orderManager = new OrderManager();
orderManager.on('orderPlaced', (order) => {
    console.log('========== ORDER RECEIPT ==========');
    console.log('Order ID:', order.id);
    console.log('Item:', order.item);
    console.log('Amount: $' + order.amount);
    console.log('==================================\n');
```

```
});

const orders = [
    { id: 101, item: 'Laptop', amount: 999.99 },
    { id: 102, item: 'Mouse', amount: 29.99 },
    { id: 103, item: 'Keyboard', amount: 79.99 }
];

orders.forEach((order) => {
    orderManager.emit('orderPlaced', order);
});

console.log('All orders processed successfully!');
```



Server.js

```
const http = require('http');

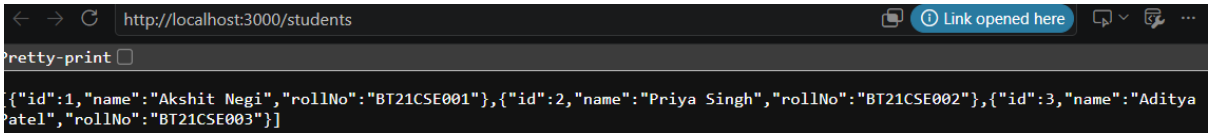
const url = require('url');

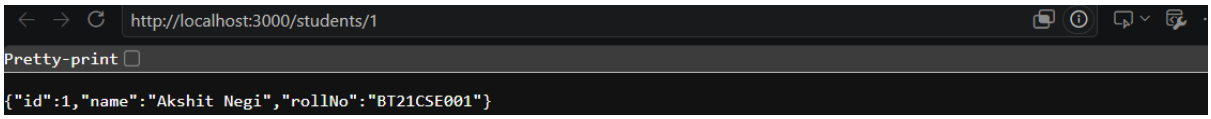
const students = [
    { id: 1, name: 'Akshit Negi', rollNo: 'BT21CSE001' },
    { id: 2, name: 'Priya Singh', rollNo: 'BT21CSE002' },
    { id: 3, name: 'Aditya Patel', rollNo: 'BT21CSE003' }
];

const server = http.createServer((req, res) => {
```

```
    const parsedUrl = url.parse(req.url, true);
    const pathname = parsedUrl.pathname;
    const query = parsedUrl.query;
    console.log(`${new Date().toISOString()} - ${req.method} ${req.url}`);
    if (pathname === '/' && req.method === 'GET') {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.end('<h1>Welcome to Node.js HTTP Server</h1><p>Use routes like /students, /students/1, /search?keyword=test</p>');
    }
    else if (pathname === '/students' && req.method === 'GET') {
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify(students));
    }
    else if (pathname.startsWith('/students/') && req.method === 'GET') {
        const id = parseInt(pathname.split('/')[2]);
        const student = students.find(s => s.id === id);
        if (student) {
            res.writeHead(200, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify(student));
        } else {
            res.writeHead(404, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ error: 'Student not found' }));
        }
    }
    else if (pathname === '/search' && req.method === 'GET') {
        const keyword = query.keyword || 'none';
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ keyword: keyword, message: `Search results for: ${keyword}` }));
```

```
    } else {
        res.writeHead(404, { 'Content-Type': 'text/html' });
        res.end('<h1>404 - Page Not Found</h1><p>The requested route does not exist.</p>');
    }
});
const PORT = 3000;
server.listen(PORT, () => {
    console.log(`\n--- Task 7: HTTP Server with Manual Routing ---`);
    console.log(`Server running at http://localhost:${PORT}/`);
    console.log('\nAvailable Routes:');
    console.log('  GET http://localhost:3000/');
    console.log('  GET http://localhost:3000/students');
    console.log('  GET http://localhost:3000/students/1');
    console.log('  GET http://localhost:3000/search?keyword=node');
    console.log('\nPress Ctrl+C to stop the server\n');
});
```







Stream-demo.js

```
const fs = require('fs');
const path = require('path');
const filePath = path.join(__dirname, 'large-file.txt');
```

```
console.log('--- Task 9: File Streams ---\n');
console.log('Creating large-file.txt with 100 lines...');
let content = '';
for (let i = 1; i <= 100; i++) {
    content += `Line ${i}: This is sample content for stream reading demonstration.\n`;
}
fs.writeFile(filePath, content, (err) => {
    if (err) throw err;
    console.log('✓ File created successfully\n');
    console.log('Reading file using fs.createReadStream...\n');
    const readStream = fs.createReadStream(filePath, { encoding: 'utf8' });
    let chunkCount = 0;
    let totalBytes = 0;
    readStream.on('data', (chunk) => {
        chunkCount++;
        totalBytes += chunk.length;
        console.log(`Chunk ${chunkCount}: ${chunk.length} bytes received`);
    });
    readStream.on('end', () => {
        console.log(`\n✓ Stream reading complete!`);
        console.log(`Total chunks: ${chunkCount}`);
        console.log(`Total bytes: ${totalBytes}`);
    });
    readStream.on('error', (err) => {
        console.error('Error reading file:', err);
    });
});
```

```
PS D:\Labassignment> node "d:\Labassignment\Full Stack\nodejs-solution\stream-demo.js"
✓ File created successfully

Reading file using fs.createReadStream...

Chunk 1: 6592 bytes received

✓ Stream reading complete!
Total chunks: 1
Total bytes: 6592
PS D:\Labassignment>
```

Id-gen.js

```
const { nanoid } = require('nanoid');

console.log('--- Task 10: Generating Unique IDs with nanoid ---\n');

console.log('Generated 5 unique IDs:\n');

for (let i = 1; i <= 5; i++) {
    const uniqueId = nanoid();
    console.log(`ID ${i}: ${uniqueId}`);
}

console.log('\n--- Custom nanoid examples ---');

console.log('Shorter ID (8 chars):', nanoid(8));

console.log('Longer ID (20 chars):', nanoid(20));

console.log('Custom alphabet:', nanoid(10, '0123456789ABCDEF'));
```

```
--- Task 10: Generating Unique IDs with nanoid ---

Generated 5 unique IDs:

ID 1: xXwCkKS7s2-x60DEDBKyp
ID 2: oDVu6YQhNupfX_whBMOHs
ID 3: OnjVkFwAXmTRJ1pEYlI21
ID 4: uXJzWhrMIKArEUgQYoxkK
ID 5: 1aQaBY2oru_dy2v_leUOD

--- Custom nanoid examples ---
Shorter ID (8 chars): N7sUpipu
Longer ID (20 chars): r69QTOGf-TPJXHH0DX1S
Custom alphabet: 7AO28VdkP2
PS D:\Labassignment>
```

calc-cli.js

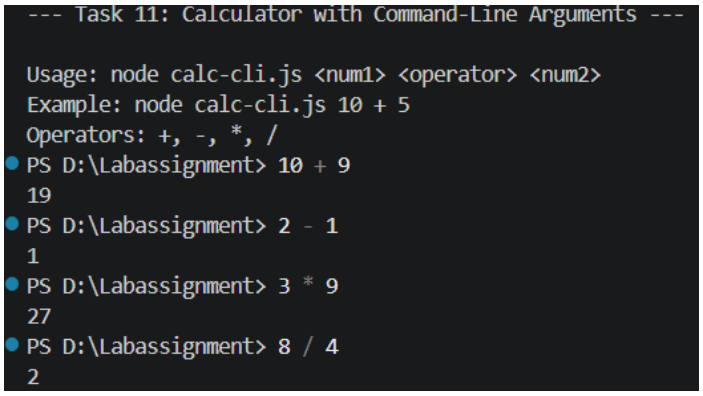
```
console.log('--- Task 11: Calculator with Command-Line Arguments ---\n');

const args = process.argv.slice(2);

if (args.length < 3) {
```

```
    console.log('Usage: node calc-cli.js <num1> <operator> <num2>');
    console.log('Example: node calc-cli.js 10 + 5');
    console.log('Operators: +, -, *, /');
    process.exit(1);
}
const num1 = parseFloat(args[0]);
const operator = args[1];
const num2 = parseFloat(args[2]);
if (isNaN(num1) || isNaN(num2)) {
    console.error('Error: Both arguments must be valid numbers');
    process.exit(1);
}
let result;
switch (operator) {
    case '+':
        result = num1 + num2;
        break;
    case '-':
        result = num1 - num2;
        break;
    case '*':
        result = num1 * num2;
        break;
    case '/':
        if (num2 === 0) {
            console.error('Error: Division by zero');
            process.exit(1);
        }
```

```
        result = num1 / num2;
        break;
    default:
        console.error('Error: Invalid operator. Use +, -, *, or /');
        process.exit(1);
}
console.log(`\n${num1} ${operator} ${num2} = ${result}`);
```



env-demo.js

```
const http = require('http');
console.log('--- Task 12: Environment Variables ---\n');
const PORT = process.env.PORT || 4000;
const server = http.createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end(`
        <h1>Welcome!</h1>
        <p>Server running on port: ${PORT}</p>
        <p>Environment Variables:</p>
        <ul>
            <li>PORT: ${process.env.PORT || 'not set (using default: 4000)'}</li>
            <li>NODE_ENV: ${process.env.NODE_ENV || 'not set'}</li>
        </ul>
```

```
  `);
});
server.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}/`);
  console.log(`PORT environment variable: ${process.env.PORT || 'not set (using default: 4000)'}`);
  console.log('\nTo run with a custom port:');
  console.log('  Linux/Mac: PORT=5000 node env-demo.js');
  console.log('  Windows: set PORT=5000 && node env-demo.js');
  console.log('\nPress Ctrl+C to stop the server\n');
});
```

